



UADY
FACULTAD DE
MATEMÁTICAS

Universidad Autónoma de Yucatán

Facultad de Matemáticas

Ingeniería de software asistida por IA

ADA 03

Context Engineering Experiment

Alumna:

Ashley Shaden Aguilar Moreno

Profesor:

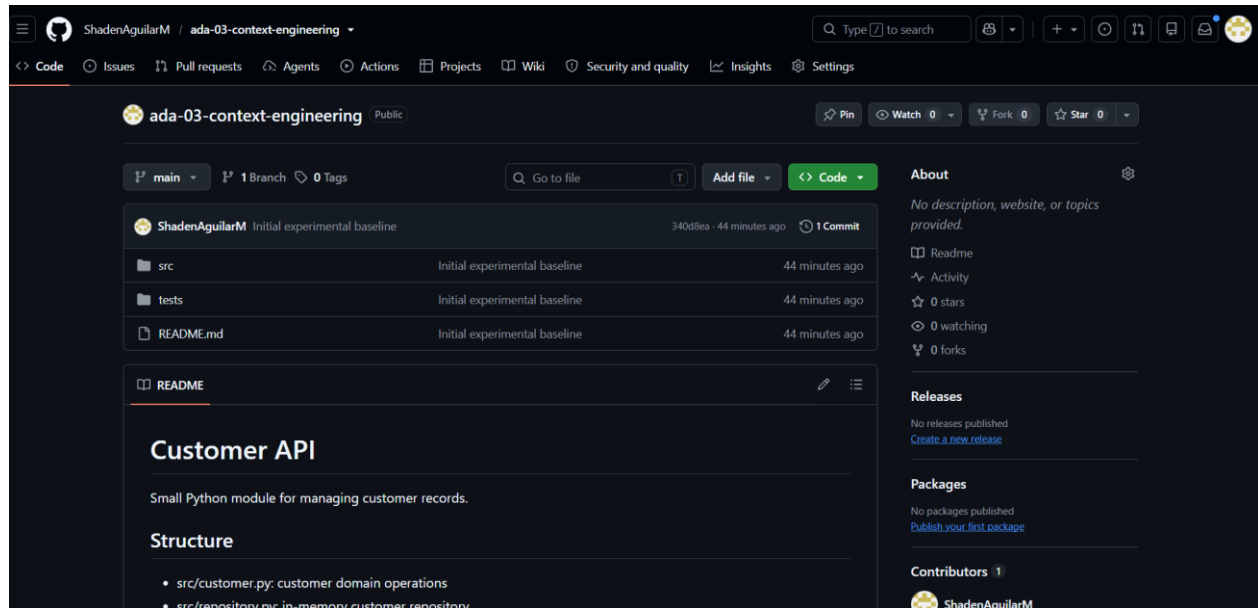
Sergio David Franco Garduño

Fecha de entrega:

10 de septiembre de 2026

1. Repositorio creado.

Link: <https://github.com/ShadenAguilarM/ada-03-context-engineering.git>



2. Línea base.

```
PS C:\Users\ashle\ada-03-context-engineering> pytest
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\ashle\ada-03-context-engineering
collected 0 items / 2 errors

===== ERRORS =====
_____ ERROR collecting tests/test_customer.py _____
ImportError while importing test module 'C:\Users\ashle\ada-03-context-engineering\tests\test_customer.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\AppData\Local\Python\pythoncore-3.14-64\Lib\importlib\_init_.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_customer.py:1: in <module>
    from src.customer import Customer, update_customer_email
E   ModuleNotFoundError: No module named 'src'

_____ ERROR collecting tests/test_repository.py _____
ImportError while importing test module 'C:\Users\ashle\ada-03-context-engineering\tests\test_repository.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\AppData\Local\Python\pythoncore-3.14-64\Lib\importlib\_init_.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_repository.py:1: in <module>
    from src.customer import Customer
E   ModuleNotFoundError: No module named 'src'

===== short test summary info =====
ERROR tests/test_customer.py
ERROR tests/test_repository.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 2 errors during collection !!!!!!!!!!!!!!!!!!!!!
===== 2 errors in 0.20s =====
PS C:\Users\ashle\ada-03-context-engineering>
```

3. Métricas

Métrica	A	B	C
Pruebas pasadas	8 Se ejecutaron exitosamente 8 pruebas al finalizar (las 4 pruebas originales del proyecto + 4 pruebas adicionales de cobertura casos límite y persistencia).	4 4 de 4 pruebas pasaron exitosamente al finalizar.	4 100% de la suite de pruebas del repositorio (test_customer.py y test_repository.py)
Pruebas fallidas	0 0 pruebas fallidas al término de la tarea.	0 pruebas fallidas al finalizar.	0 Cero pruebas fallidas al finalizar.
Requisitos cumplidos	7/7 Cumplió los 7, pero hizo cambios adicionales innecesarios, principalmente al modificar/agregar pruebas.	7/7 Cumplió los 7 haciendo únicamente los cambios necesarios.	7/7 Cumplió los 7, SPEC.md hacía explícitos esos requisitos y AGENTS.md le indicaba minimizar cambios y no modificar tests.
Archivos modificados	5 4 archivos modificados (customer.py, repository.py, test_customer.py, test_repository.py) y 1 creado (pytest.ini).	3 Modificados: customer.py y repository.py. Creado: pytest.ini (configuración de ruta).	3 customer.py y repository.py y se añadió pytest.ini para la resolución de rutas de pytest.
Cambios innecesarios	2 El agente terminó con 8 tests pasando, pero originalmente había 4. El agente añadió 4 tests adicionales: dos en test_customer.py y dos en test_repository.py	0 Se aplicó la solución mínima necesaria sin modificar tests preexistentes ni agregar dependencias extras.	0 No se modificaron tests preexistentes, no se agregaron librerías externas y se preservaron las interfaces públicas según AGENTS.md.
Iteraciones	4 1. Diagnóstico e inspección inicial de pruebas. 2. Creación de pytest.ini para resolver el ModuleNotFoundError en ejecución directa. 3. Implementación de lógica en dominio y repositorio (4 pruebas pasadas). 4. Ampliación de casos de prueba y verificación final completa (8 pruebas pasadas).	1 Se resolvió en una sola iteración (diagnóstico previo → implementación → verificación exitosa al primer intento).	1 Se diagnosticó, implementó y validó en un único ciclo de trabajo directo sin requerir retrabajos o correcciones adicionales.
Intervenciones humanas	0 El flujo completo (análisis, codificación, configuración y verificación) se ejecutó de forma 100% autónoma.	0 Ejecución 100% autónoma de extremo a extremo sin requerir aclaraciones ni pausas al usuario.	0 Todo el flujo se completó de manera autónoma sin requerir asistencia o aclaraciones intermedias.
Problemas introducidos	Ninguna regresión; se preservó la firma de métodos, la inmutabilidad de identificadores y el contrato de	0 Sin regresiones ni rotura de	0 Cero regresiones o incompatibilidades

	excepciones esperado.	contratos existentes.	generadas.
Tiempo	3 minutos y 30 segundos.	2 minutos y 25 segundos.	1 minuto y 35 segundos.
Score /10	$30 + 20 + 8 + 15 + 10 + 10 = 93/100 = 9.3$	$30 + 20 + 15 + 15 + 10 + 10 = 100/100 = 10$	$30 + 20 + 15 + 15 + 10 + 10 = 100/100 = 10$

4. Preguntas de análisis.

1. ¿Cuál fue tu hipótesis?

Mi hipótesis fue que, al proporcionar al agente un contexto más completo, estructurado y relevante, su desempeño mejoraría. Esperaba que el contexto del repositorio permitiera reducir la exploración innecesaria y que el contexto diseñado mediante SPEC.md y AGENTS.md ayudara al agente a comprender con mayor precisión los requisitos y restricciones de la tarea. Por lo tanto, esperaba obtener soluciones correctas con menos errores, menos cambios innecesarios y menor esfuerzo.

2. ¿Cuál experimento produjo el mejor resultado y por qué?

El experimento C produjo el mejor resultado general. Aunque A, B y C lograron cumplir los requisitos funcionales, C contó con un contexto diseñado específicamente para orientar al agente. SPEC.md definió claramente los siete requisitos y criterios de aceptación, mientras que AGENTS.md estableció reglas sobre cómo realizar los cambios y verificarlos. Esto permitió que el agente trabajara de manera más dirigida, en menos tiempo y evitara modificaciones innecesarias.

3. ¿Qué errores aparecieron en A y no en C?

En A se presentó una mayor exploración del repositorio y se realizaron cambios adicionales, incluyendo modificaciones y pruebas nuevas que no eran necesarias para cumplir la tarea. También tuvo que investigar por sí mismo información que en C estaba explícitamente definida. En C, las instrucciones indicaban realizar el cambio mínimo, no modificar las pruebas y verificar los resultados antes y después de la implementación. Por ello, no se observaron los mismos cambios innecesarios.

4. ¿Qué información del repositorio fue más útil?

La información más útil fueron las pruebas existentes y la separación de responsabilidades entre customer.py y repository.py. Las pruebas permitieron identificar comportamientos específicos, como convertir el correo a minúsculas, lanzar `ValueError("invalid-email")`, preservar customer_id y created_by, actualizar updated_by y manejar el caso de un cliente inexistente. Además, repository.py mostró que la actualización debía comprobar la existencia del cliente, delegar la operación y guardar el resultado.

5. ¿Qué aportó SPEC.md?

SPEC.md convirtió los requisitos implícitos en requisitos explícitos. Definió los siete comportamientos que debía cumplir la solución y estableció criterios de aceptación concretos. Esto redujo la necesidad de que el agente tuviera que inferir el comportamiento únicamente a partir del código y las pruebas. Por ejemplo, especificó directamente que el correo debía estar en minúsculas, que debía validarse su sintaxis y que customer_id y created_by debían conservarse.

6. ¿Qué función tuvo AGENTS.md?

AGENTS.md estableció cómo debía trabajar el agente, no solamente qué debía implementar. Indicó inspeccionar primero el repositorio, realizar el cambio mínimo, no modificar las pruebas, preservar las interfaces existentes y ejecutar las pruebas antes y después de los cambios. Por lo tanto, SPEC.md respondió principalmente a qué debe cumplir la solución, mientras que AGENTS.md orientó cómo debe trabajar el agente.

7. ¿Más contexto significa necesariamente mejor contexto?

No. El experimento muestra que la cantidad de información por sí sola no garantiza un mejor resultado. Lo importante es que el contexto sea relevante, claro y esté bien estructurado. El contexto adicional de C fue útil porque estaba organizado en requisitos, criterios de aceptación y reglas de trabajo. En cambio, proporcionar información innecesaria podría aumentar la exploración del agente o distraerlo de la tarea principal.

8. ¿Qué información fue redundante?

Parte de la información de SPEC.md era equivalente a lo que ya podía inferirse de las pruebas existentes. Por ejemplo, las pruebas ya indicaban el comportamiento esperado para el lowercase, el formato del correo, la conservación de los datos de auditoría y el manejo de clientes inexistentes. Por ello, SPEC.md no siempre agregó información completamente nueva, sino que principalmente hizo explícitos y organizó comportamientos que ya estaban presentes en el repositorio.

9. ¿Qué intervención humana fue necesaria?

No fue necesaria una intervención humana para corregir la implementación funcional del agente. Las intervenciones realizadas durante la preparación del experimento estuvieron relacionadas principalmente en aceptar cambios o procedimientos del agente.

10. ¿Qué cambiarías en SPEC.md y AGENTS.md?

En SPEC.md agregaría ejemplos más explícitos de entradas válidas e inválidas y aclararía cualquier comportamiento límite de la validación de correos. En AGENTS.md agregaría una instrucción para comparar explícitamente los archivos modificados contra la línea base antes de finalizar. También indicaría que el agente debe reportar de forma resumida los archivos modificados, las pruebas ejecutadas y cualquier desviación respecto a la especificación. Estos cambios permitirían mejorar todavía más la trazabilidad y facilitarían la evaluación del resultado.

11. ¿Qué aprendiste sobre la responsabilidad del desarrollador al usar agentes?

Aprendí que utilizar un agente de código no elimina la responsabilidad del desarrollador. El desarrollador debe proporcionar un contexto adecuado, definir claramente los requisitos, establecer restricciones, revisar los cambios realizados y verificar que la solución sea correcta. El experimento mostró que un agente puede producir una solución funcional, pero también puede realizar cambios innecesarios si no se le proporcionan suficientes restricciones. Por ello, el desarrollador debe aprender a diseñar el contexto y no depender únicamente de un prompt. También debe validar el resultado mediante pruebas y revisión del código.

5. Pregunta final.

En máximo 200 palabras: ¿por qué un desarrollador que utiliza agentes de código necesita aprender Context Engineering y no solamente mejores prompts?

Un desarrollador que utiliza agentes de código necesita aprender Context Engineering porque el rendimiento de un agente no depende únicamente de qué tan bien esté escrito el prompt, sino de qué información recibe, cómo se estructura y qué herramientas y restricciones tiene disponibles.

Un buen prompt puede indicar claramente qué hacer, pero el Context Engineering permite diseñar el entorno necesario para que el agente lo haga correctamente. Esto incluye seleccionar archivos relevantes del repositorio, proporcionar instrucciones, definir restricciones, incluir ejemplos, controlar el estado de la tarea y evitar información innecesaria que pueda confundir al agente.

Además, los agentes pueden analizar código, ejecutar comandos, modificar archivos y tomar decisiones de manera autónoma. Por ello, proporcionarles el contexto adecuado puede reducir errores, disminuir la intervención humana y mejorar la calidad de sus resultados.

En conclusión, prompt engineering optimiza la instrucción, mientras que context engineering optimiza todo el entorno de información en el que el agente toma decisiones. Para desarrollar software con agentes de forma confiable y reproducible, ambas habilidades son necesarias.